Programación De Computadores II

Parcial 3
Sistema de Gestión Bibliotecaria (SGB): Diagramas UML

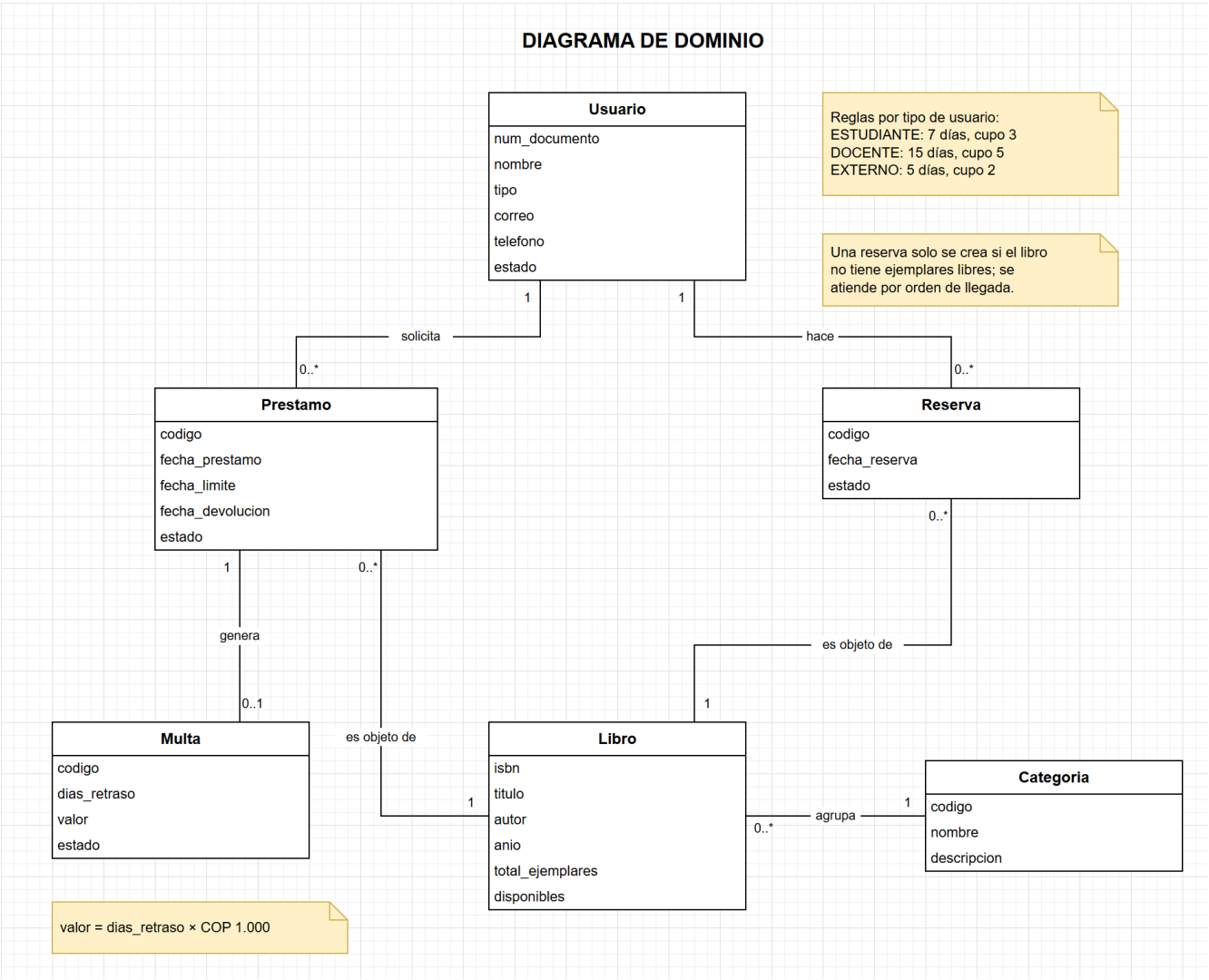
Carlos Santiago Merchan Centeno 01251151009
Daniel Fernando Vesga Tarazona 01251151006

Universidad De Santander

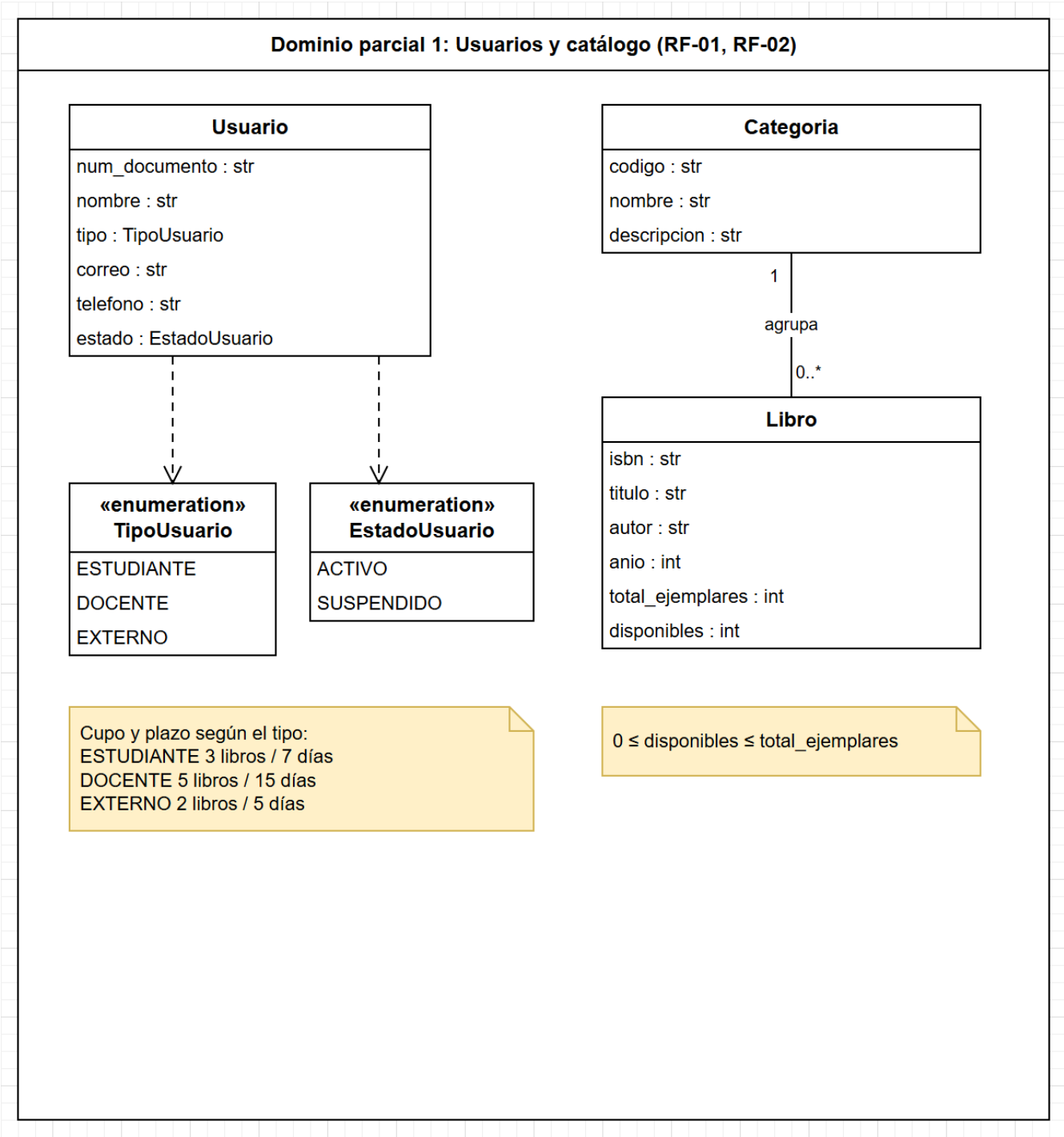
9/11/2026

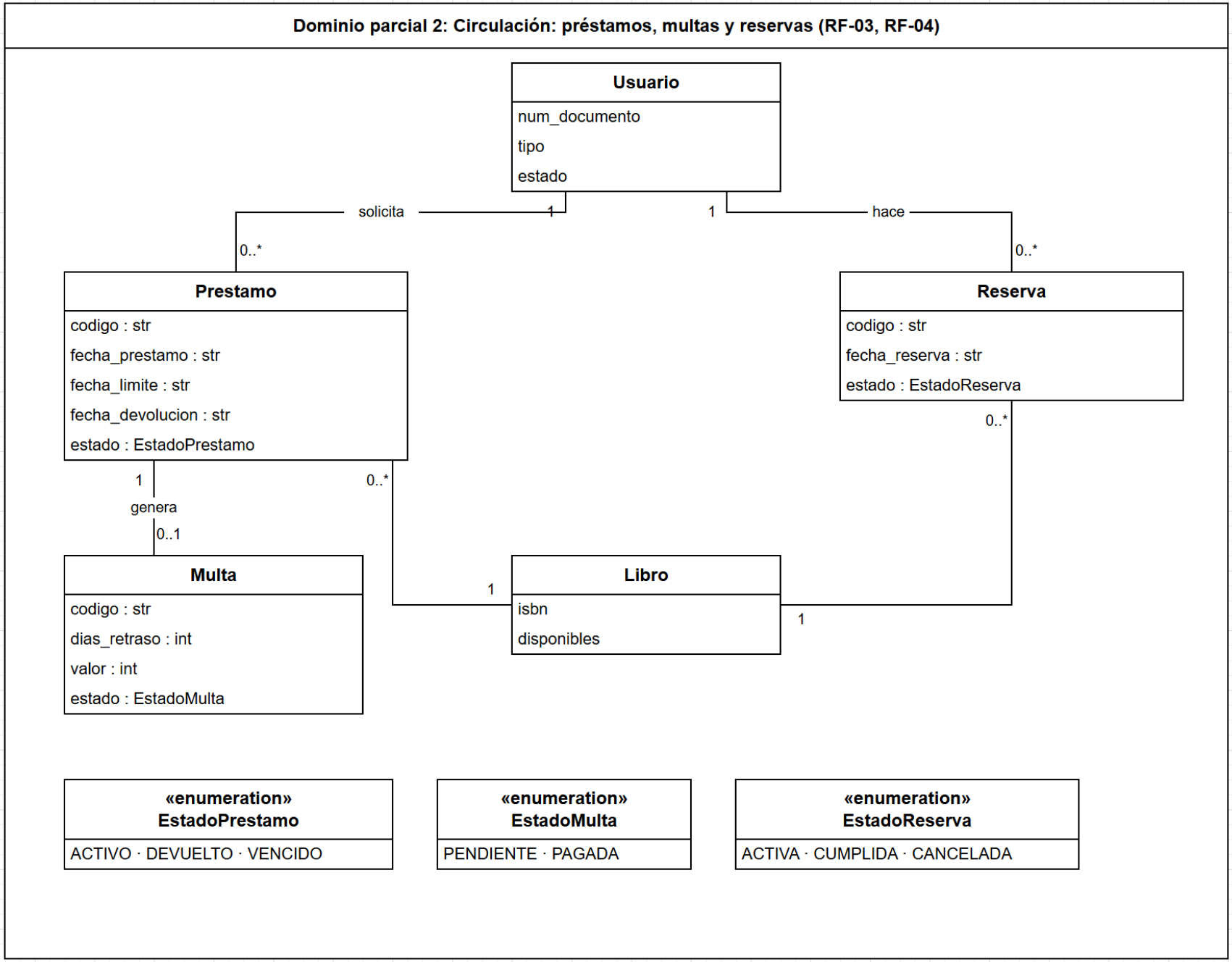
DISEÑO ESTRUCTURAL.

Diagrama de dominio:



Diagramas de dominio parcial:




Dominio parcial 2: Circulación: préstamos, multas y reservas (RF-03, RF-04)
Usuario
num_documento
tipo
estado
solicita
1
1
hace
0..*
0..*
Prestamo
codigo : str
fecha_prestamo : str
fecha_limite : str
fecha_devolucion : str
estado : EstadoPrestamo
Reserva
codigo : str
fecha_reserva : str
estado : EstadoReserva
0..*
1
genera
0..1
0..*
Multa
codigo : str
dias_retraso : int
valor : int
estado : EstadoMulta
1
Libro
isbn
disponibles
1
«enumeration»
EstadoPrestamo
ACTIVO · DEVUELTO · VENCIDO
«enumeration»
EstadoMulta
PENDIENTE · PAGADA
«enumeration»
EstadoReserva
ACTIVA · CUMPLIDA · CANCELADA

Dominio parcial 3: Análisis de datos (RF-05)

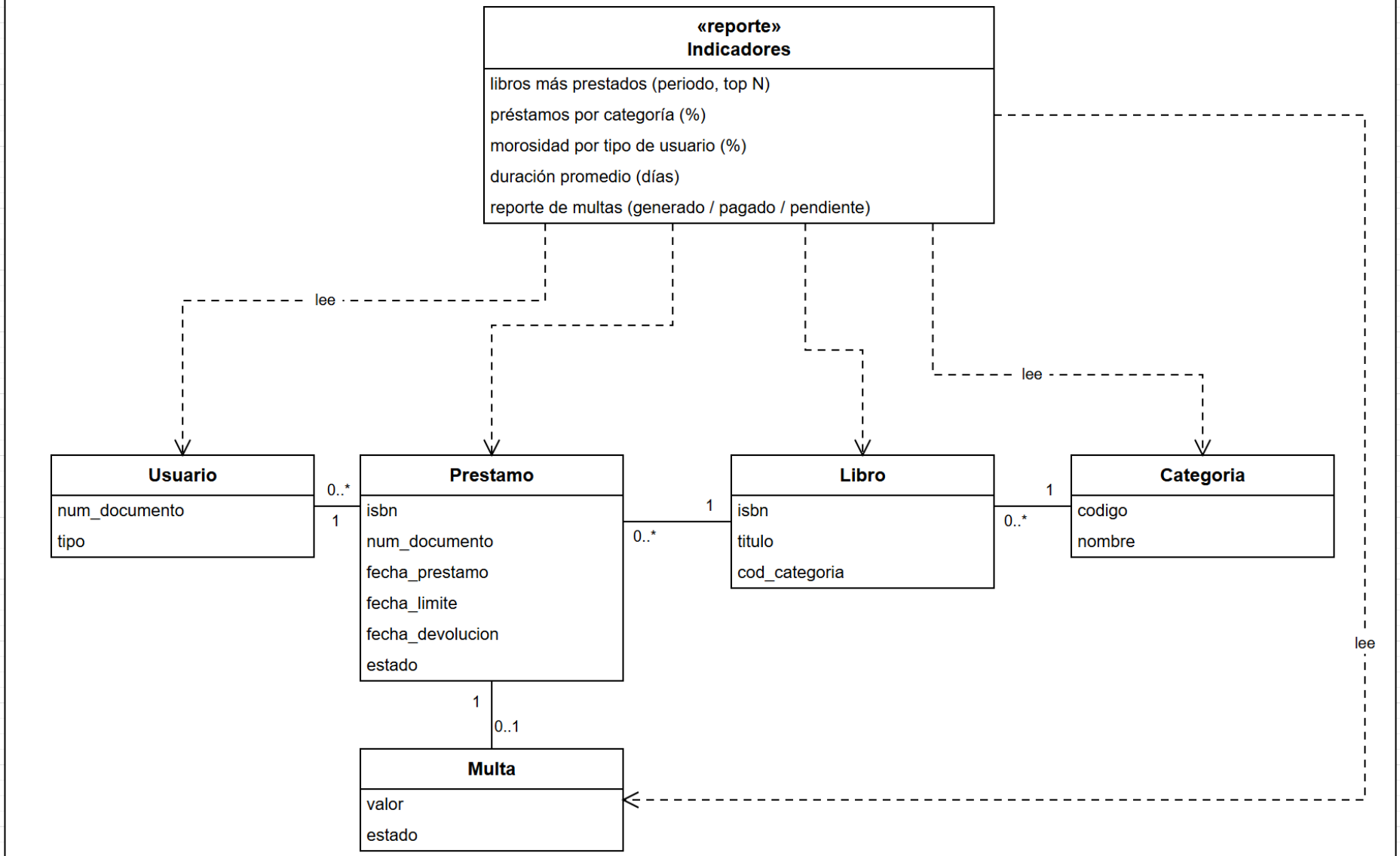


Diagrama de clases:

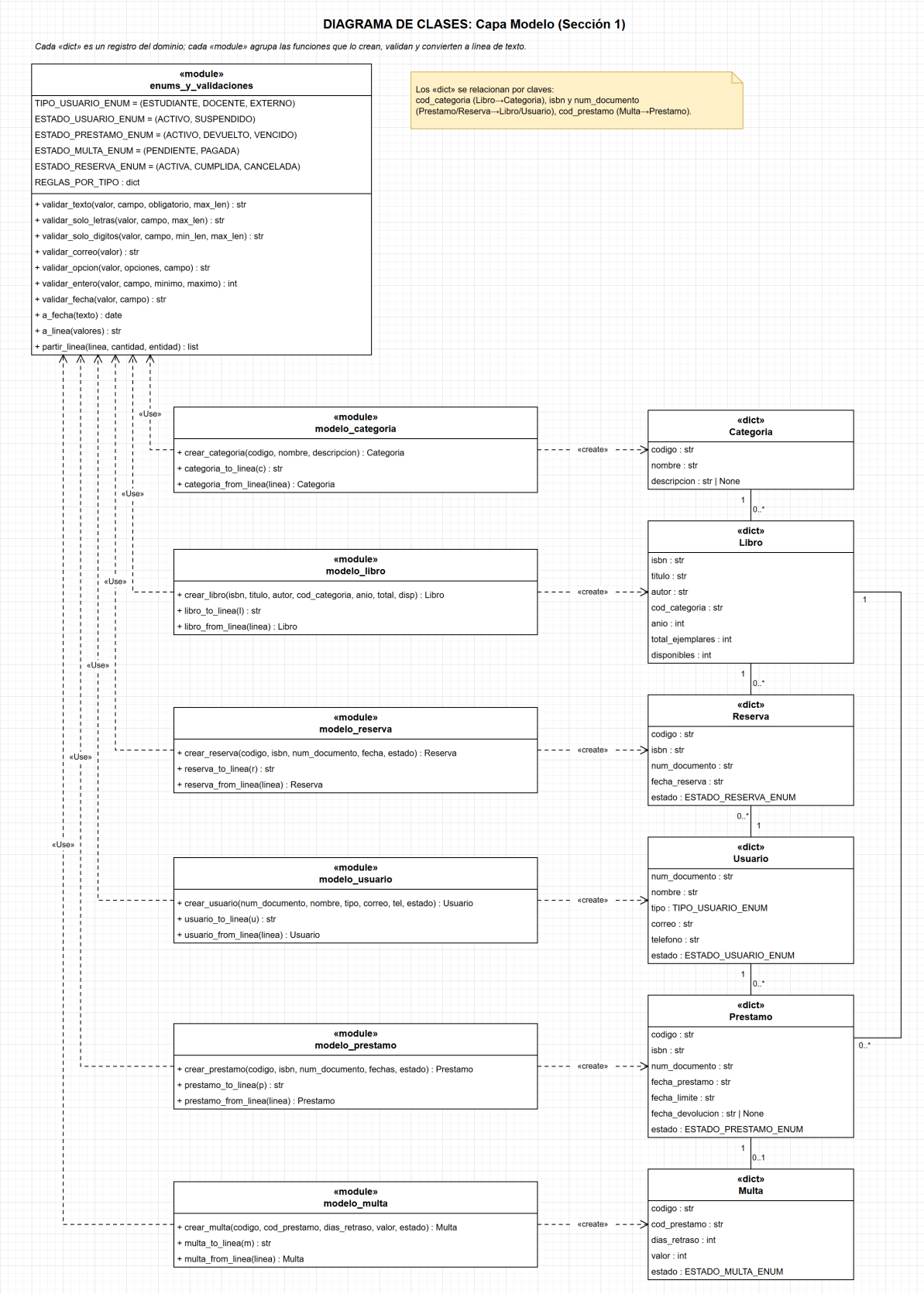
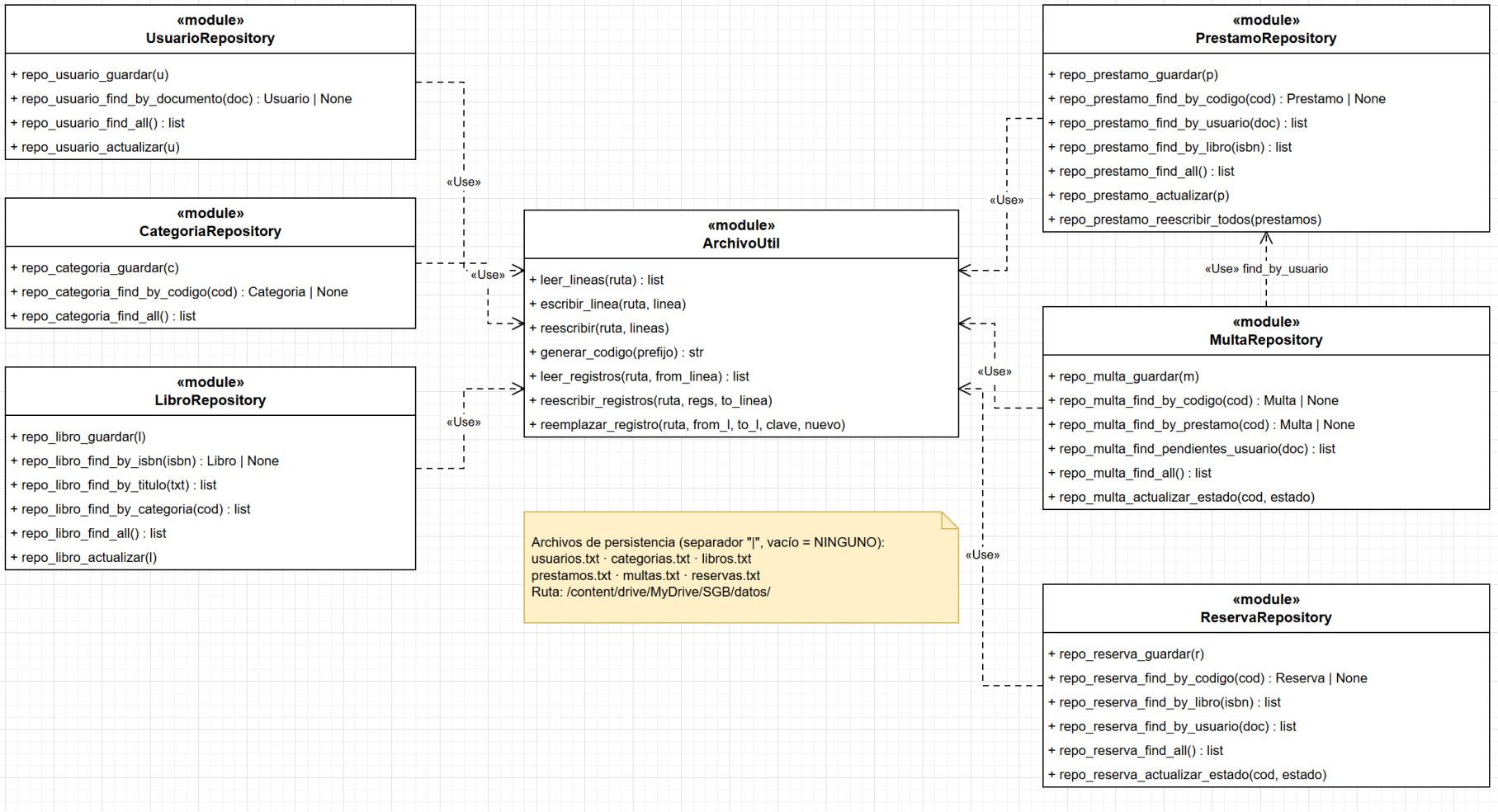


DIAGRAMA DE CLASES: Capa Repositorios (Sección 2)



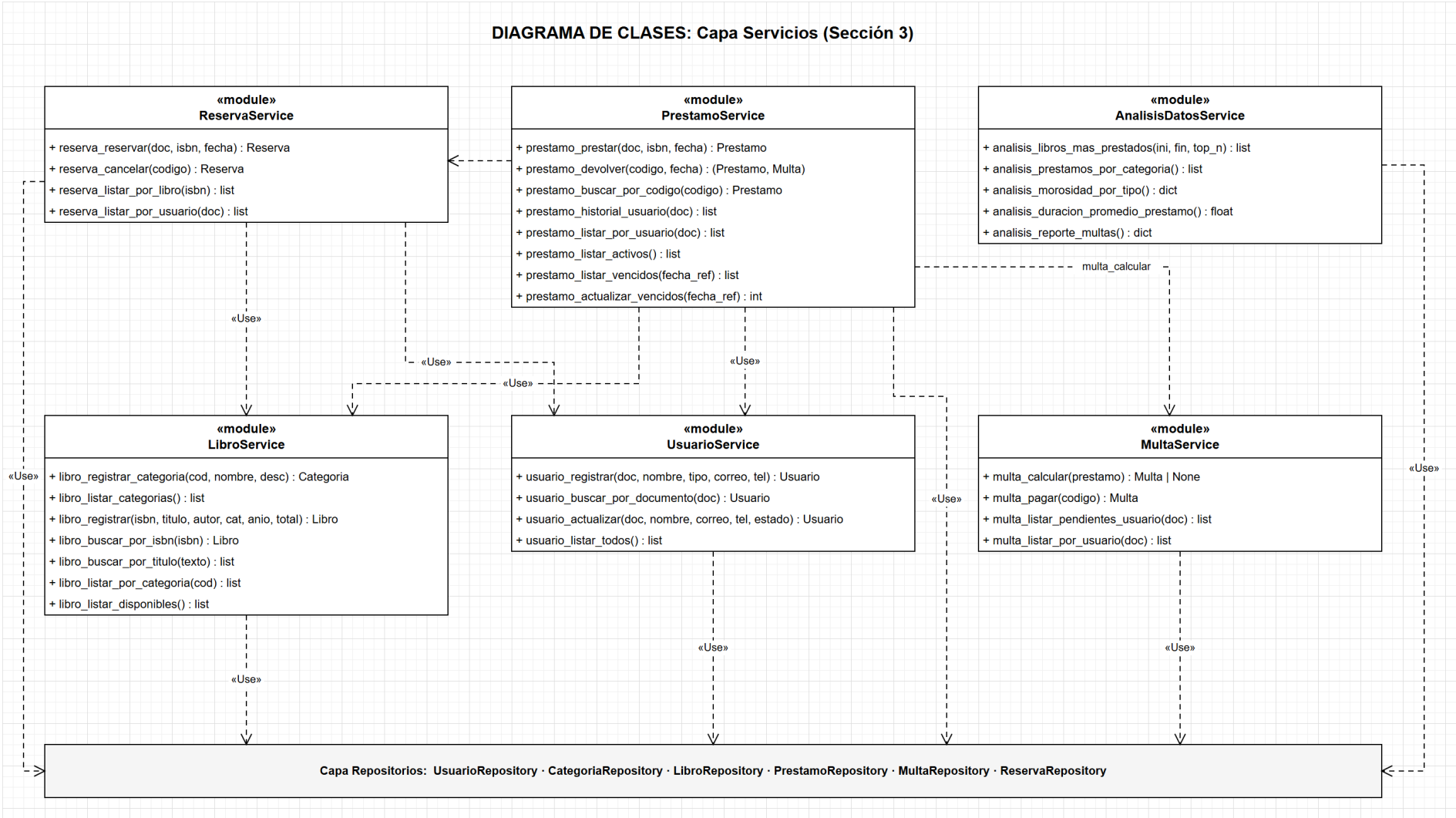

DIAGRAMA DE CLASES: Capa Servicios (Sección 3)
«module»
ReservaService
+ reserva_reservar(doc, isbn, fecha) : Reserva
+ reserva_cancelar(codigo) : Reserva
+ reserva_listar_por_libro(isbn) : list
+ reserva_listar_por_usuario(doc) : list
«module»
PrestamoService
+ prestamo_prestar(doc, isbn, fecha) : Prestamo
+ prestamo_devolver(codigo, fecha) : (Prestamo, Multa)
+ prestamo_buscar_por_codigo(codigo) : Prestamo
+ prestamo_historial_usuario(doc) : list
+ prestamo_listar_por_usuario(doc) : list
+ prestamo_listar_activos() : list
+ prestamo_listar_vencidos(fecha_ref) : list
+ prestamo_actualizar_vencidos(fecha_ref) : int
«module»
AnalisisDatosService
+ analisis_libros_mas_prestados(ini, fin, top_n) : list
+ analisis_prestamos_por_categoria() : list
+ analisis_morosidad_por_tipo() : dict
+ analisis_duracion_promedio_prestamo() : float
+ analisis_reporte_multas() : dict
multa_calcular
«Use»
«Use»
«Use»
«Use»
«Use»
«module»
LibroService
+ libro_registrar_categoria(cod, nombre, desc) : Categoria
+ libro_listar_categorias() : list
+ libro_registrar(isbn, titulo, autor, cat, anio, total) : Libro
+ libro_buscar_por_isbn(isbn) : Libro
+ libro_buscar_por_titulo(texto) : list
+ libro_listar_por_categoria(cod) : list
+ libro_listar_disponibles() : list
«module»
UsuarioService
+ usuario_registrar(doc, nombre, tipo, correo, tel) : Usuario
+ usuario_buscar_por_documento(doc) : Usuario
+ usuario_actualizar(doc, nombre, correo, tel, estado) : Usuario
+ usuario_listar_todos() : list
«module»
MultaService
+ multa_calcular(prestamo) : Multa | None
+ multa_pagar(codigo) : Multa
+ multa_listar_pendientes_usuario(doc) : list
+ multa_listar_por_usuario(doc) : list
«Use»
«Use»
«Use»
«Use»
«Use»
«Use»
«Use»
«Use»
Capa Repositorios: UsuarioRepository · CategoriaRepository · LibroRepository · PrestamoRepository · MultaRepository · ReservaRepository

DIAGRAMA DE CLASES: Capa Interfaz de usuario (Sección 4)

Todas las funciones ui_* piden los datos con leer_formulario(): validación inmediata de cada campo, "<" vuelve al campo anterior, "*" cancela y el resumen final permite corregir cualquier campo antes de guardar.

«module»
Main (Celda 5.1)

+ menu_principal()

«Use»

«module»
consola_y_formularios

ATRAS = "<"

CANCELAR = "*"

+ leer_formulario(titulo, campos, confirmar) : dict | None
+ leer_campo(c, valores, n, total, previo)
+ campo(clave, etiqueta, validar, opciones, por_defecto, opcional)
+ elegir_opcion(texto, opciones)
+ v_documento_nuevo / v_documento_existente / v_documento_habilitado
+ v_isbn_nuevo / v_isbn_existente / v_codigo(prefijo, ...)
+ v_letras / v_texto / v_digitos / v_correo / v_entero / v_fecha
+ leer_texto / leer_entero / leer_fecha / leer_opcion
+ ejecutar_menu(titulo, acciones)
+ imprimir_tabla(encabezados, filas)
+ mostrar_usuarios / libros / prestamos / multas / reservas
+ limpiar_pantalla()

1

«module»
menu_usuarios

+ menu_usuarios()
- ui_registrar_usuario()
- ui_consultar_usuario()
- ui_actualizar_contacto()
- ui_cambiar_estado()

«Use»

2

«module»
menu_libros

+ menu_libros()
- ui_registrar_categoria()
- ui_listar_categorias()
- ui_registrar_libro()
- ui_consultar_isbn()
- ui_buscar_titulo()
- ui_listar_por_categoria()

«Use»

3

«module»
menu_prestamos

+ menu_prestamos()
- ui_prestar()
- ui_devolver()
- ui_consultar_prestamo()
- ui_prestamos_usuario()
- ui_vencidos()

«Use»

4

«module»
menu_reservas_multas

+ menu_reservas_multas()
- ui_reservar()
- ui_cancelar_reserva()
- ui_cola_reservas()
- ui_multas_pendientes()
- ui_pagar_multa()
- ui_historial_usuario()

«Use»

5

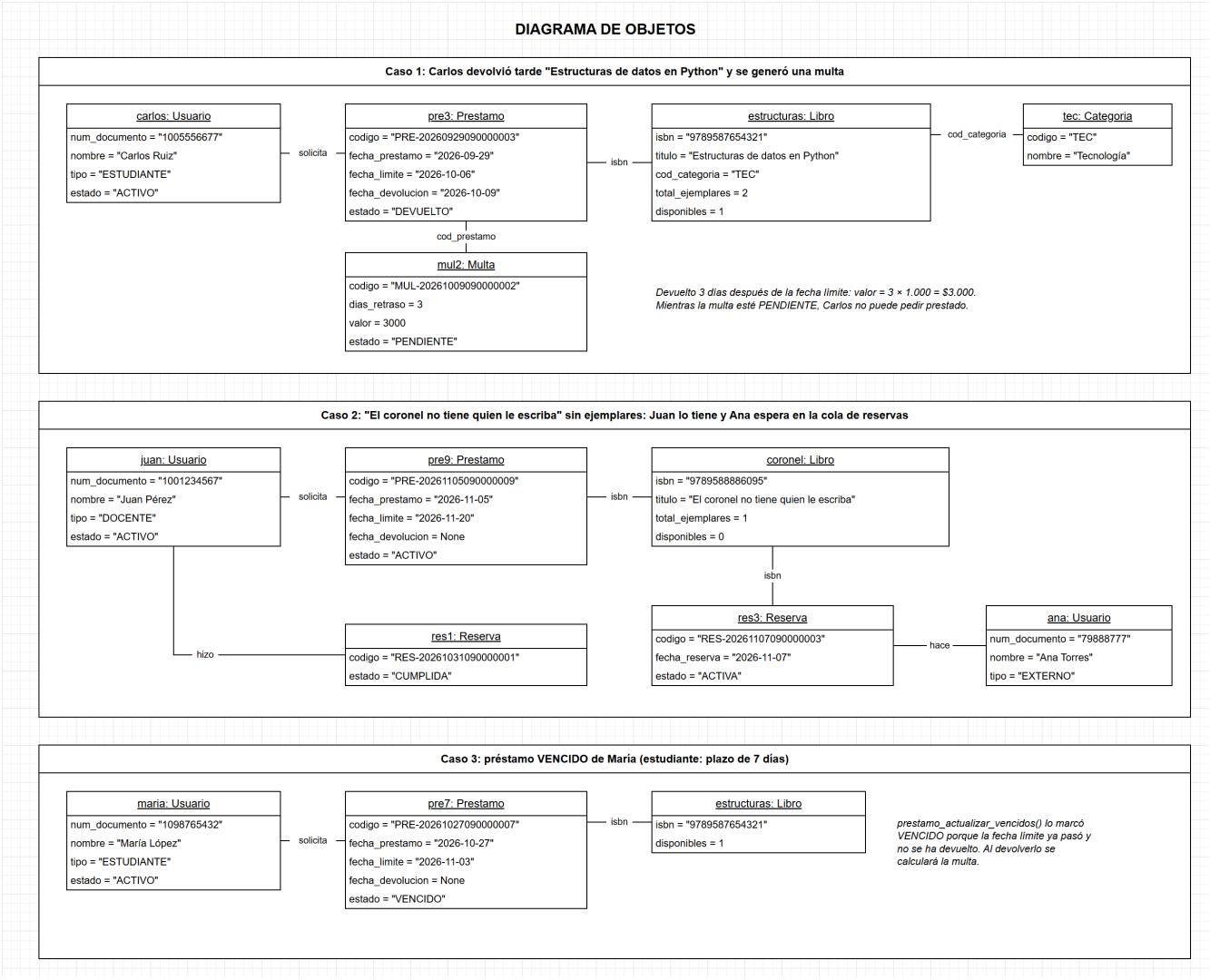
«module»
menu_analisis

+ menu_analisis()
- ui_libros_mas_prestados()
- ui_prestamos_por_categoria()
- ui_morosidad()
- ui_duracion_promedio()
- ui_reporte_multas()

«Use»

Capa Servicios: UsuarioService · LibroService · PrestamoService · ReservaService · MultaService · AnalisisDatosService

Diagrama de objetos:



DISEÑO DE COMPORTAMIENTO:

Diagrama estructurado:

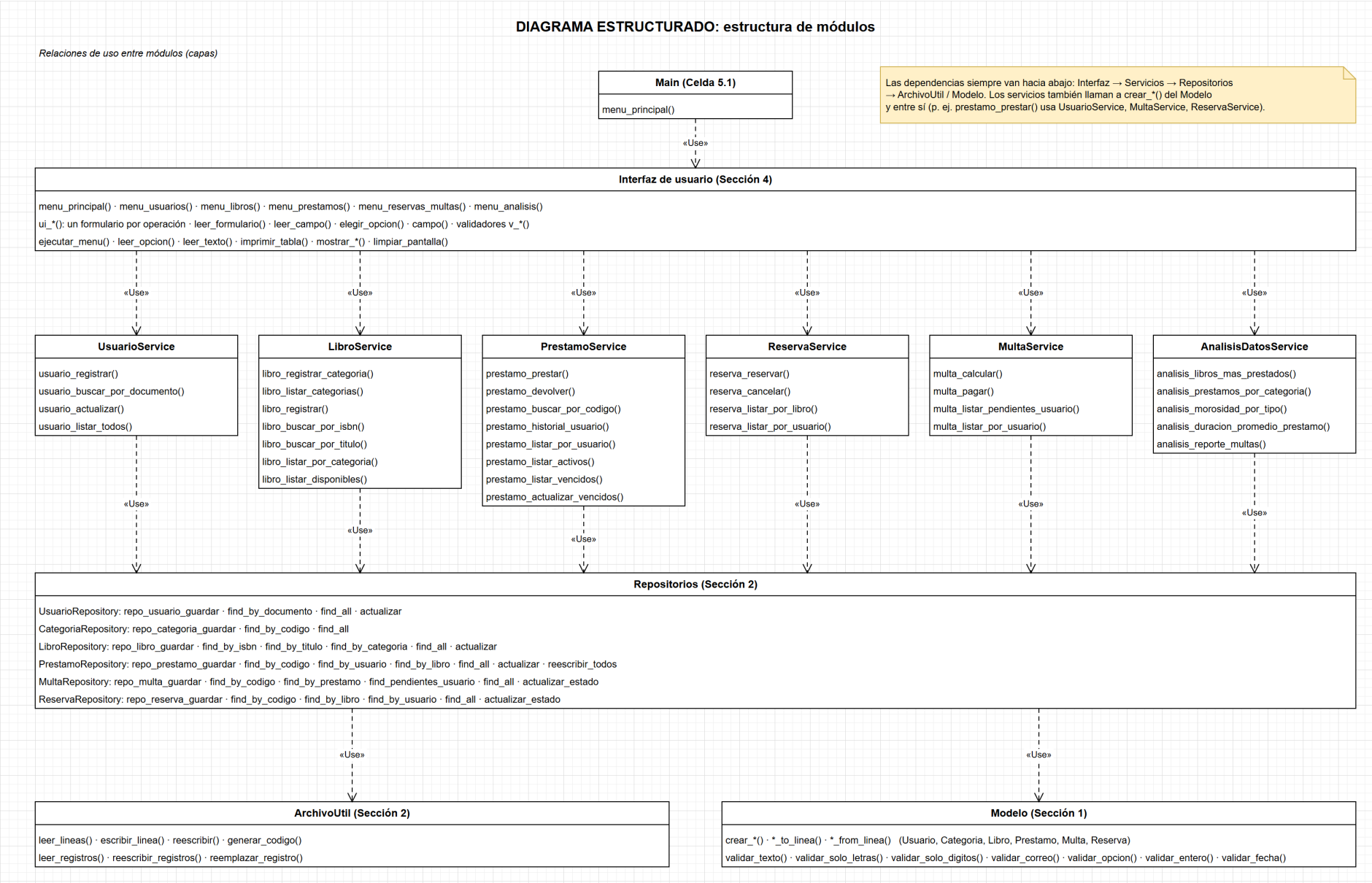
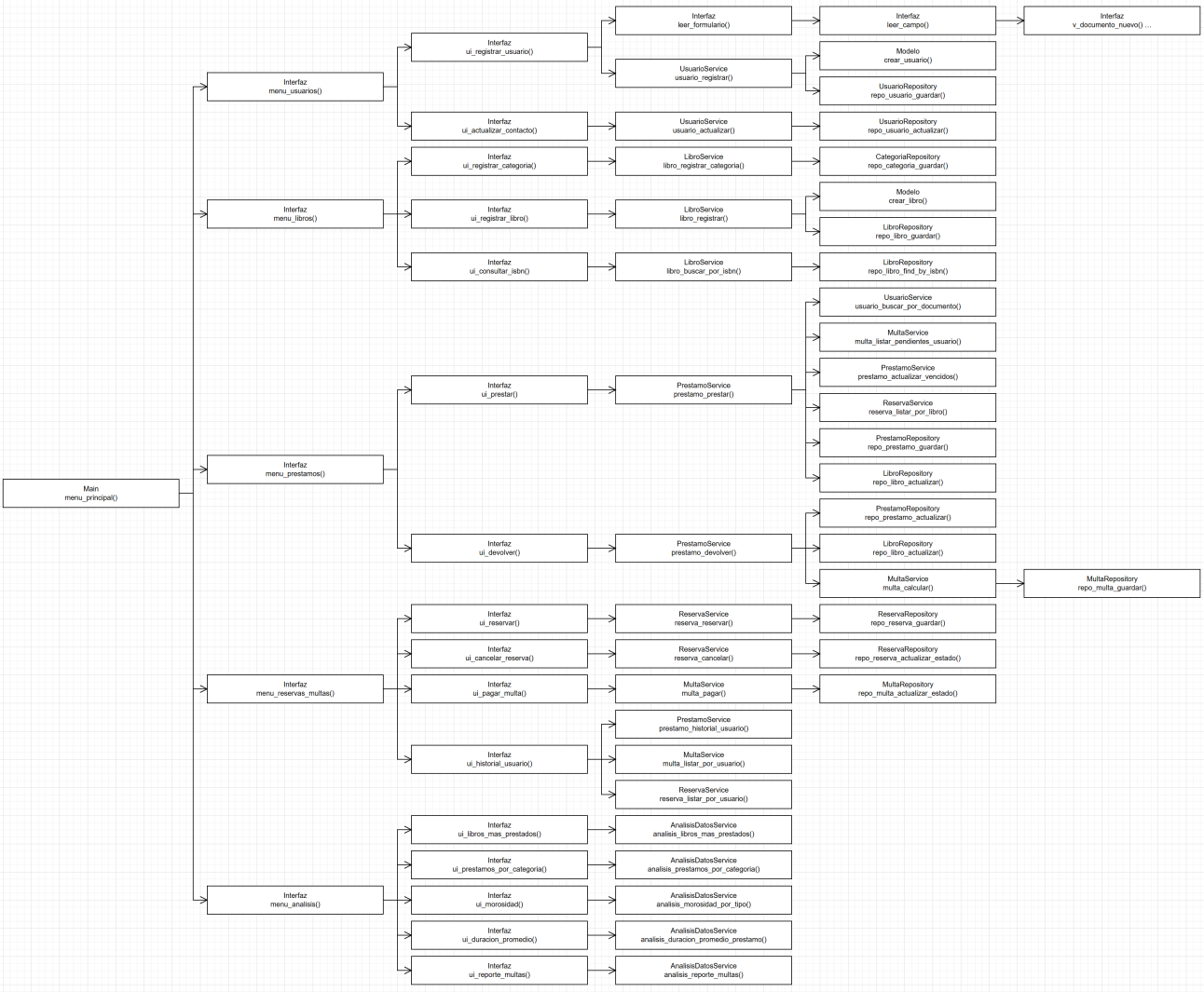


DIAGRAMA ESTRUCTURADO: estructura de llamadas entre funciones

Cada caja indica Módulo / función(); la flecha va de la función que llama a la llamada.



Persistencia: cómo un repositorio actualiza un registro en el archivo .txt

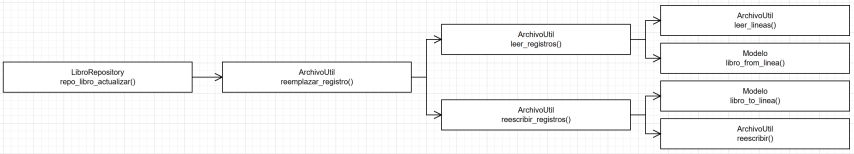


DIAGRAMA ESTRUCTURADO: diseño de cada función (pseudocódigo) 1/2

Módulo Main Función menu_principal()
limpiar_pantalla()
prestamo_actualizar_vencidos()
ejecutar_menu("SGB — MENÚ PRINCIPAL", {1: menu_usuarios, 2: menu_libros,
3: menu_prestamos, 4: menu_reservas_multas, 5: menu_analisis})
MOSTRAR "Hasta pronto."

Módulo Interfaz Función leer_campo(c, valores, numero, total, previo)
opciones = c.opciones ; defecto = previo o c.por_defecto
Si: hay opciones entonces MOSTRAR la lista numerada
REPETIR
texto = LEER "[numero/total] etiqueta [defecto]: "
Si: texto = "<" o texto = "*" entonces ENTREGAR texto
Si: texto vacío:
Si: hay defecto entonces texto = defecto
Si No: Si c.opcional entonces ENTREGAR ""
Si No: MOSTRAR "Este campo es obligatorio" y REPETIR
INTENTAR:
Si: hay opciones entonces texto = elegir_opcion(texto, opciones)
ENTREGAR c.validar(texto, valores)
Si ocurre ValueError e: MOSTRAR "✗ " + e + " Corrija el dato"
HASTA QUE el dato sea válido

Módulo UsuarioService Función usuario_registrar(doc, nombre, tipo, correo, telefono)
u = crear_usuario(doc, nombre, tipo, correo, telefono)
Si: repo_usuario_find_by_documento(u.num_documento) ≠ nulo
LANZAR ERROR "Ya existe un usuario con ese documento"
repo_usuario_guardar(u)
ENTREGAR u

Módulo Interfaz Función ejecutar_menu(titulo, acciones)
MIENTRAS verdadero:
op = leer_opcion(titulo, opciones)
Si: op = "0" entonces ENTREGAR
Si: op NO está en acciones entonces MOSTRAR "Opción no válida" y CONTINUAR
INTENTAR acciones[op]()
Si ocurre ValueError e: MOSTRAR "ERROR: " + e

Módulo Interfaz Función elegir_opcion(texto, opciones)
Si: texto es número y 1 <= texto <= cantidad de opciones
ENTREGAR opciones[texto - 1]
PARA CADA v EN opciones:
Si: MAYÚSCULAS(texto) = MAYÚSCULAS(v) entonces ENTREGAR v
LANZAR ERROR "no es una opción válida"

Módulo Modelo Función validar_solo_letras(valor, campo, max_len)
texto = validar_texto(valor, campo, max_len) (no vacío, sin "|")
Si: el primer carácter no es letra o algún carácter no es letra, espacio, . ' -
LANZAR ERROR "solo admite letras (sin números ni símbolos)"
ENTREGAR texto

Módulo Modelo Función crear_usuario(num_documento, nombre, tipo, correo, telefono, estado)
ENTREGAR diccionario con:
num_documento = validar_solo_digitos(num_documento, 5, 12)
nombre = validar_solo_letras(nombre)
tipo = validar_opcion(tipo, TIPO_USUARIO_ENUM)
correo = validar_correo(correo)
telefono = validar_solo_digitos(telefono, 7, 10)
estado = validar_opcion(estado, ESTADO_USUARIO_ENUM)

Módulo Interfaz Función leer_formulario(titulo, campos, confirmar): dict
valores = diccionario vacío ; i = 0
REPETIR
MIENTRAS i < cantidad de campos:
r = leer_campo(campos[i], valores, i + 1, total, valores[clave])
Si: r = "*" entonces MOSTRAR "Operación cancelada" y ENTREGAR nulo
Si: r = "<" entonces i = i - 1 (si i = 0 avisa "ya está en el primer campo")
Si No: valores[campos[i].clave] = r ; i = i + 1
Si: confirmar = falso entonces ENTREGAR valores
MOSTRAR resumen numerado de los campos
resp = LEER "¿Guardar? (s / número / < / *)"
Si: resp = "s" entonces ENTREGAR valores
Si: resp = "*" entonces ENTREGAR nulo
Si: resp = "<" entonces i = último campo
Si: resp es un número de campo entonces i = resp - 1
HASTA QUE se guarde o se cancele

Módulo Modelo Función validar_solo_digitos(valor, campo, min_len, max_len)
texto = validar_texto(valor, campo) SIN espacios
Si: texto no está formado solo por dígitos 0-9
LANZAR ERROR "solo admite números"
Si: longitud(texto) < min_len o longitud(texto) > max_len
LANZAR ERROR "debe tener entre min_len y max_len dígitos"
ENTREGAR texto

Módulo LibroService Función libro_registrar(isbn, titulo, autor, cod_categoria, anio, total)
l = crear_libro(isbn, titulo, autor, cod_categoria, anio, total)
Si: repo_libro_find_by_isbn(l.isbn) ≠ nulo entonces LANZAR ERROR "ISBN repetido"
Si: repo_categoria_find_by_codigo(l.cod_categoria) = nulo
LANZAR ERROR "No existe la categoría"
repo_libro_guardar(l)
ENTREGAR l

DIAGRAMA ESTRUCTURADO: diseño de cada función (pseudocódigo) 2/2

Módulo PrestamoService Función prestamo_prestar(num_documento, isbn, fecha): Prestamo
Si: fecha > hoy entonces LANZAR ERROR "La fecha no puede ser futura"
usuario = usuario_buscar_por_documento(num_documento)
Si: usuario.estado ≠ ACTIVO entonces LANZAR ERROR "SUSPENDIDO"
Si: multa_listar_pendientes_usuario(doc) no está vacía
LANZAR ERROR "tiene multas pendientes"
libro = libro_buscar_por_isbn(isbn)
prestamo_actualizar_vencidos()
abiertos = préstamos del usuario con estado ≠ DEVUELTO
Si: longitud(abiertos) >= cupo del tipo entonces LANZAR ERROR "cupo"
Si: ya tiene un ejemplar de ese libro entonces LANZAR ERROR
Si: libro.disponibles <= 0 entonces LANZAR ERROR "puede reservarlo"
cola = reserva_listar_por_libro(isbn) (por orden de llegada)
Si: el usuario está en la cola en la posición pos:
Si: pos >= libro.disponibles entonces LANZAR ERROR "reservas anteriores"
repo_reserva_actualizar_estado(su reserva, CUMPLIDA)
Si No: Si longitud(cola) >= libro.disponibles
LANZAR ERROR "ejemplares apartados por reservas"
p = NUEVO préstamo(generar_codigo("PRE"), isbn, doc, fecha, fecha + días del tipo)
repo_prestamo_guardar(p)
libro.disponibles = libro.disponibles - 1 ; repo_libro_actualizar(libro)
ENTREGAR p

Módulo AnalisisDatosService Función analisis_morosidad_por_tipo(): dict
prestamo_actualizar_vencidos()
PARA CADA tipo EN TIPO_USUARIO_ENUM:
prestamos = préstamos de usuarios de ese tipo
morosos = préstamos VENCIDOS o devueltos después de la fecha límite
tasa = morosos / total × 100 (0 si no hay préstamos)
resultado[tipo] = (total, morosos, tasa)
ENTREGAR resultado

Módulo ArchivoUtil Función leer_lineas(ruta): list
INTENTAR abrir ruta en UTF-8
ENTREGAR las líneas no vacías sin espacios finales
Si el archivo no existe: ENTREGAR lista vacía

Módulo PrestamoService Función prestamo_devolver(codigo, fecha): (Prestamo, Multa)
p = prestamo_buscar_por_codigo(codigo)
Si: p.estado = DEVUELTO entonces LANZAR ERROR "ya fue devuelto"
Si: fecha < p.fecha_prestamo o fecha > hoy entonces LANZAR ERROR
p.fecha_devolucion = fecha ; p.estado = DEVUELTO
repo_prestamo_actualizar(p)
libro.disponibles = MIN(libro.disponibles + 1, libro.total_ejemplares)
repo_libro_actualizar(libro)
ENTREGAR (p, multa_calcular(p))

Módulo PrestamoService Función prestamo_actualizar_vencidos(fecha_ref): int
cambios = 0
PARA CADA p EN repo_prestamo_find_all():
Si: p.estado = ACTIVO y p.fecha_limite < fecha_ref
p.estado = VENCIDO ; cambios = cambios + 1
Si: cambios > 0 entonces repo_prestamo_reescribir_todos(prestamos)
ENTREGAR cambios

Módulo AnalisisDatosService Función analisis_libros_mas_prestados(fecha_ini, fecha_fin, top_n)
Si: fecha_ini > fecha_fin entonces LANZAR ERROR
titulos = {isbn: titulo} de repo_libro_find_all()
contador = Counter(isbn de cada préstamo con fecha_ini <= fecha_prestamo <= fecha_fin)
ENTREGAR [(isbn, titulos[isbn], veces) PARA CADA par EN contador.most_common(top_n)]

Módulo AnalisisDatosService Funciones de prestamos_por_categoria, duración y multas
por_categoria: Counter(categoría del libro de cada préstamo) y % sobre el total
duracion_promedio: promedio de (fecha_devolucion - fecha_prestamo) de los DEVUELTOS
reporte_multas: generado = suma de todas; pagado = suma PAGADA; pendiente = suma PENDIENTE

Módulo ArchivoUtil Función generar_codigo(prefijo): str
ts = fecha y hora actual "AAAAMMDDhhmmss" + milisegundos (17 dígitos)
Si: ts <= último código generado entonces ts = último + 1
último = ts
ENTREGAR prefijo + "-" + ts

Módulo MultaService Función multa_calcular(prestamo): Multa
dias = prestamo.fecha_devolucion - prestamo.fecha_limite
Si: dias <= 0 entonces ENTREGAR nulo
Si: repo_multa_find_by_prestamo(prestamo.codigo) ≠ nulo entonces ENTREGAR esa multa
m = NUEVA multa(generar_codigo("MUL"), prestamo.codigo, dias, dias × 1000, PENDIENTE)
repo_multa_guardar(m)
ENTREGAR m

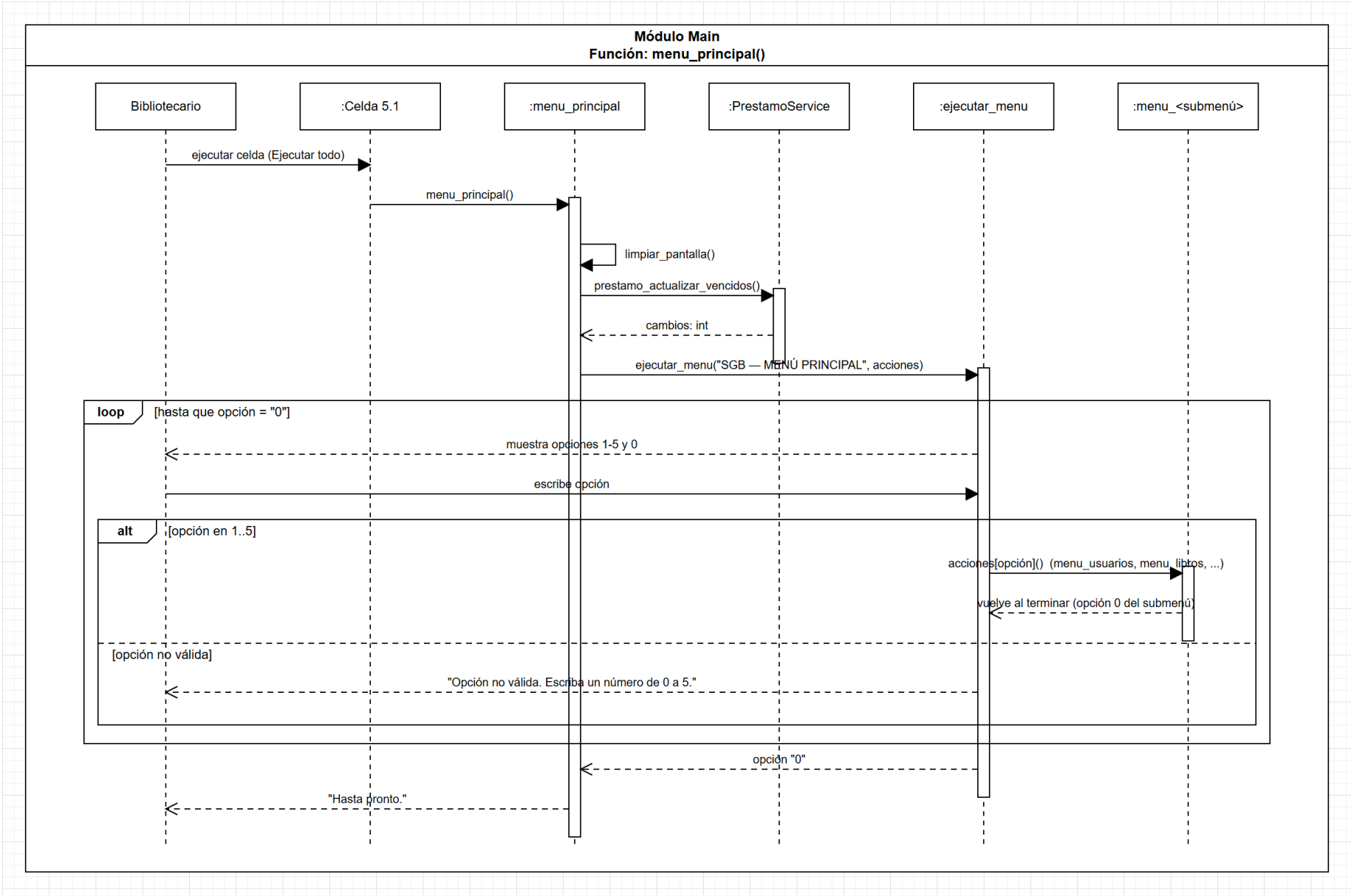
Módulo MultaService Función multa_pagar(codigo): Multa
m = repo_multa_find_by_codigo(codigo)
Si: m = nulo entonces LANZAR ERROR "No existe la multa"
Si: m.estado = PAGADA entonces LANZAR ERROR "ya estaba pagada"
repo_multa_actualizar_estado(codigo, PAGADA)
ENTREGAR m

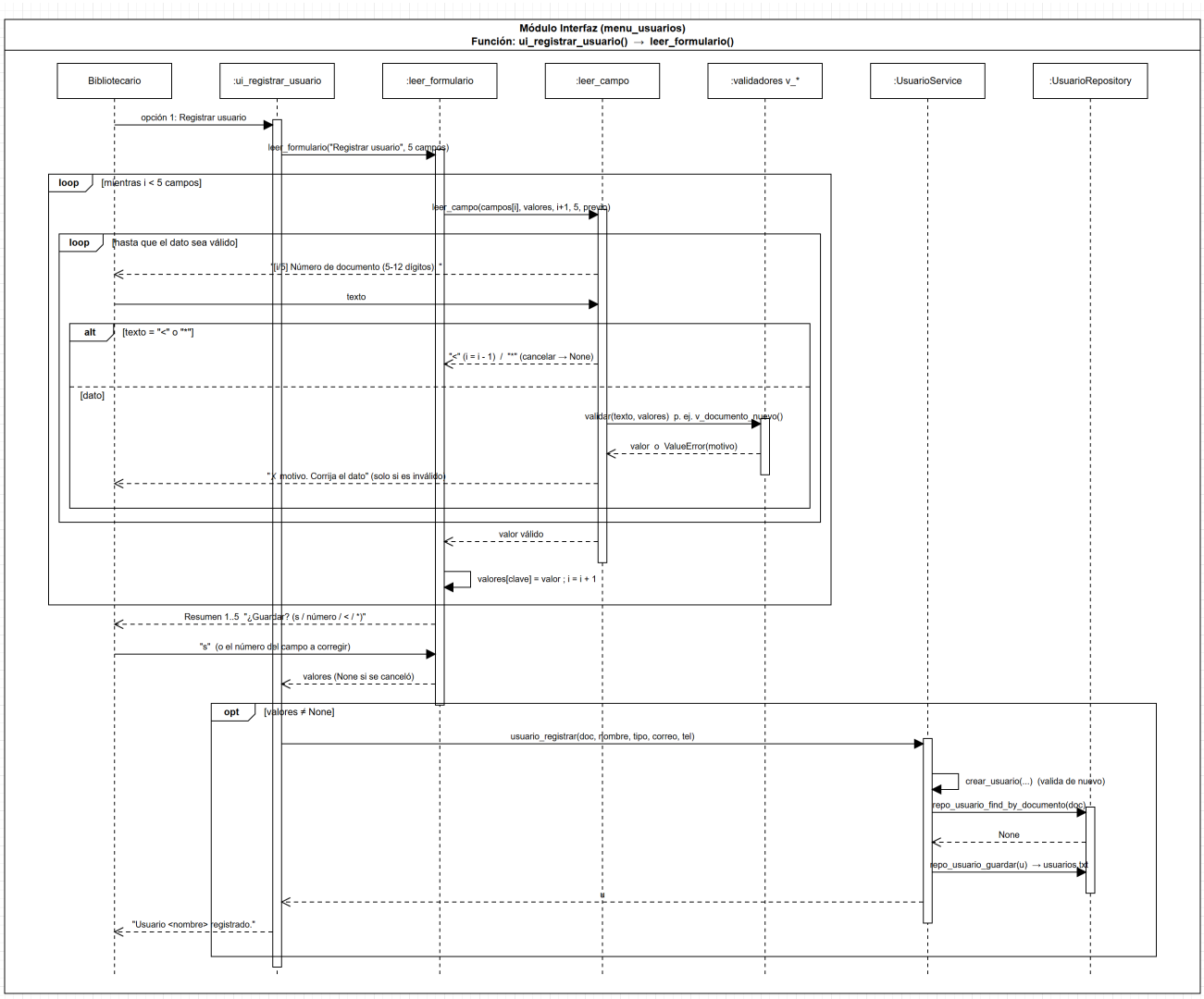
Módulo ReservaService Función reserva_reservar(num_documento, isbn, fecha): Reserva
usuario = usuario_buscar_por_documento(num_documento)
Si: usuario.estado ≠ ACTIVO entonces LANZAR ERROR
libro = libro_buscar_por_isbn(isbn) ; cola = reserva_listar_por_libro(isbn)
Si: libro.disponibles > longitud(cola)
LANZAR ERROR "tiene ejemplares: puede prestarlo"
Si: el usuario ya está en la cola o ya tiene el libro entonces LANZAR ERROR
r = NUEVA reserva(generar_codigo("RES"), isbn, doc, fecha, ACTIVA)
repo_reserva_guardar(r)
ENTREGAR r

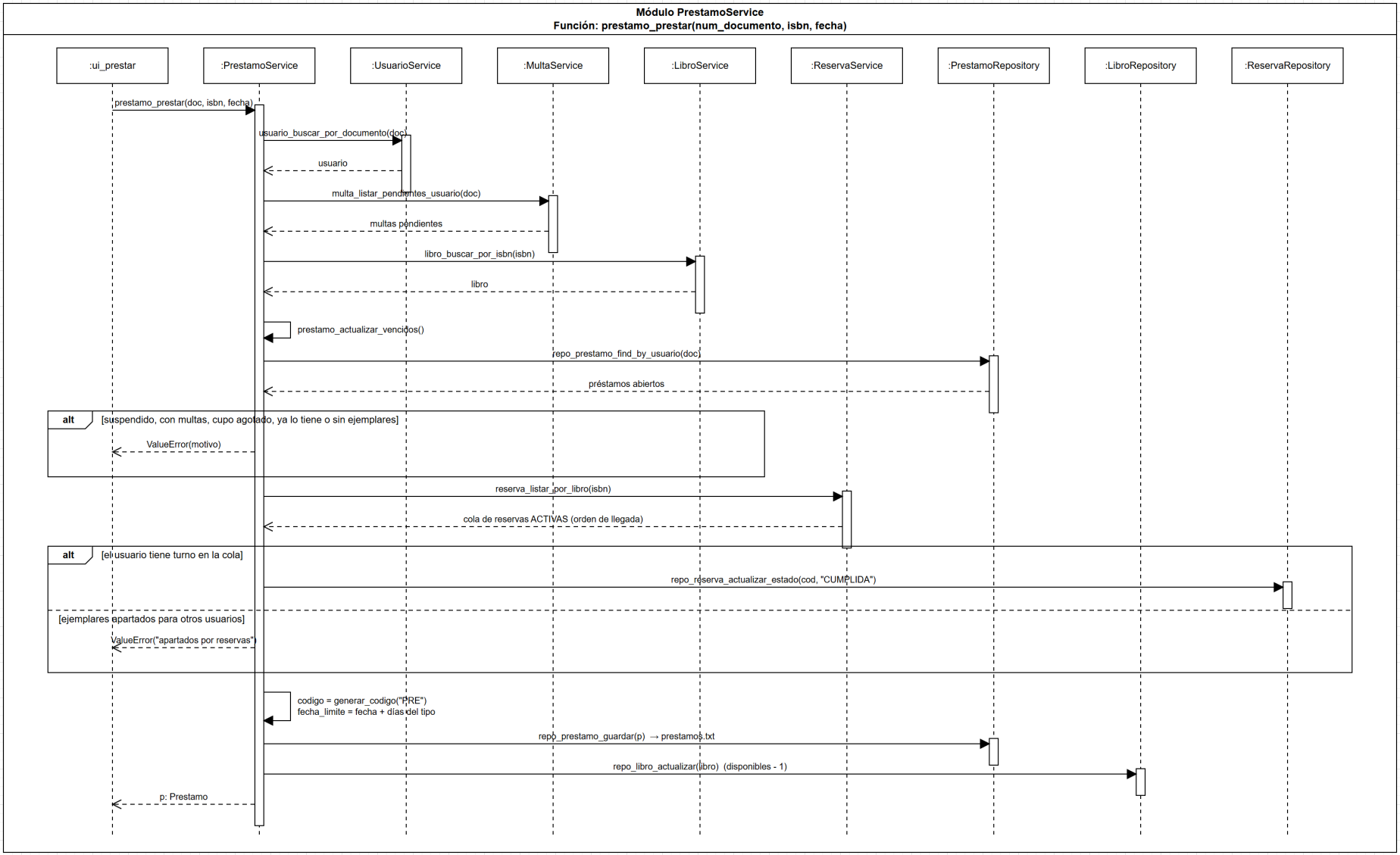
Módulo ArchivoUtil Función reemplazar_registro(ruta, from_linea, to_linea, clave, nuevo)
registros = leer_registros(ruta, from_linea)
PARA CADA i, r EN registros:
Si: r[clave] = nuevo[clave]
registros[i] = nuevo
reescribir_registros(ruta, registros, to_linea)
ENTREGAR
LANZAR ERROR "No existe el registro"

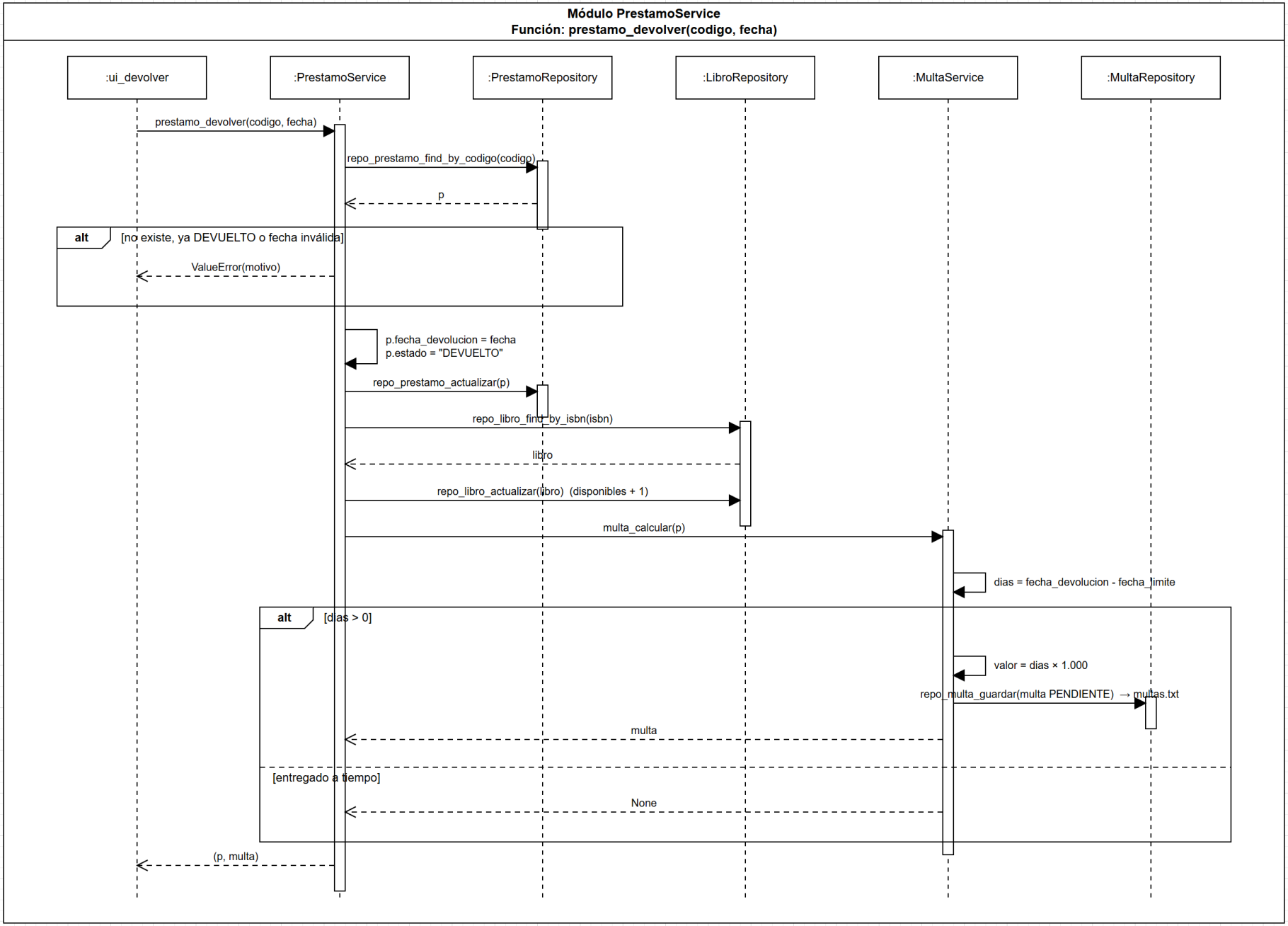
DISEÑO DE INTERACCION:

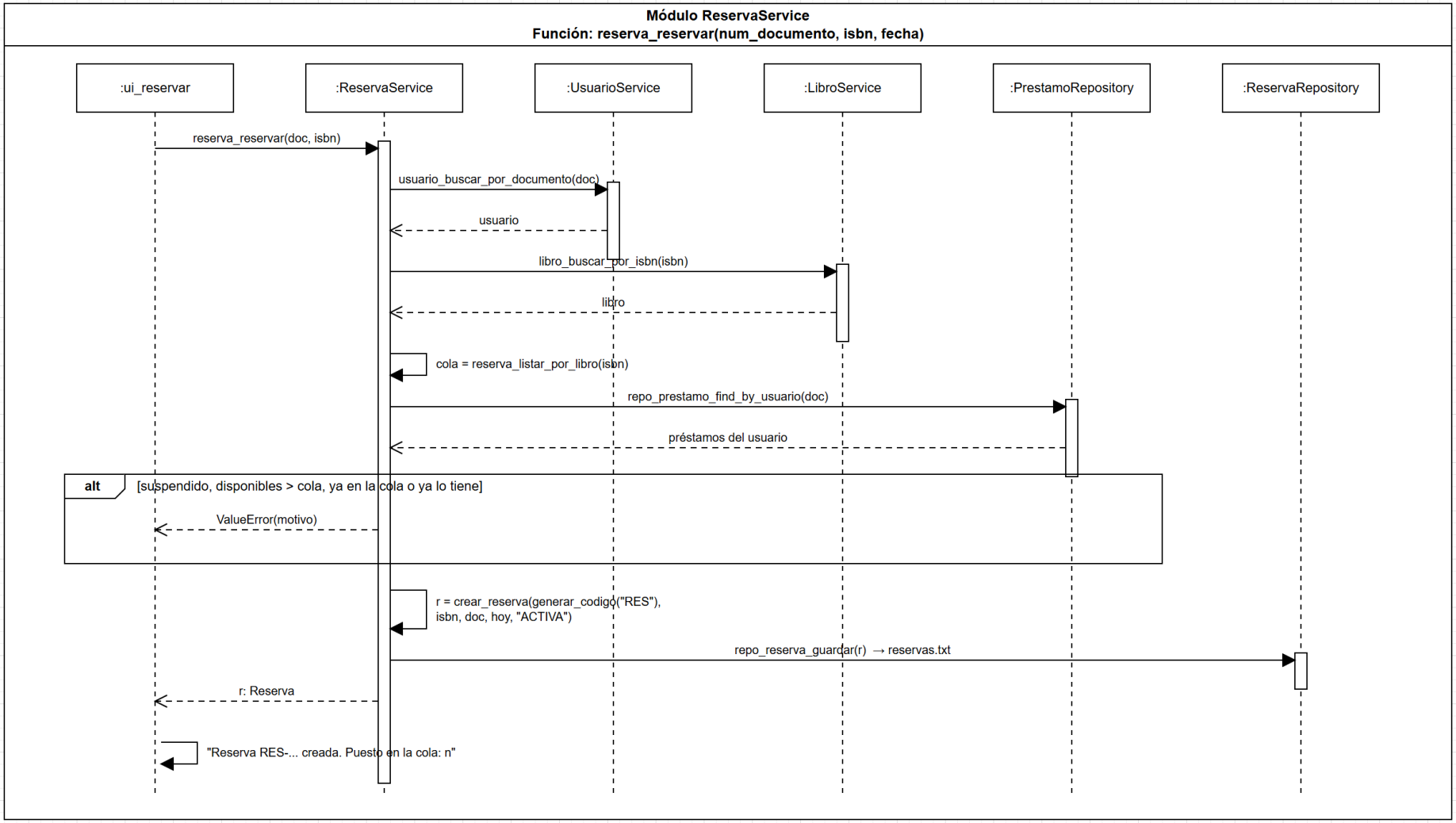
Diagrama de secuencias:

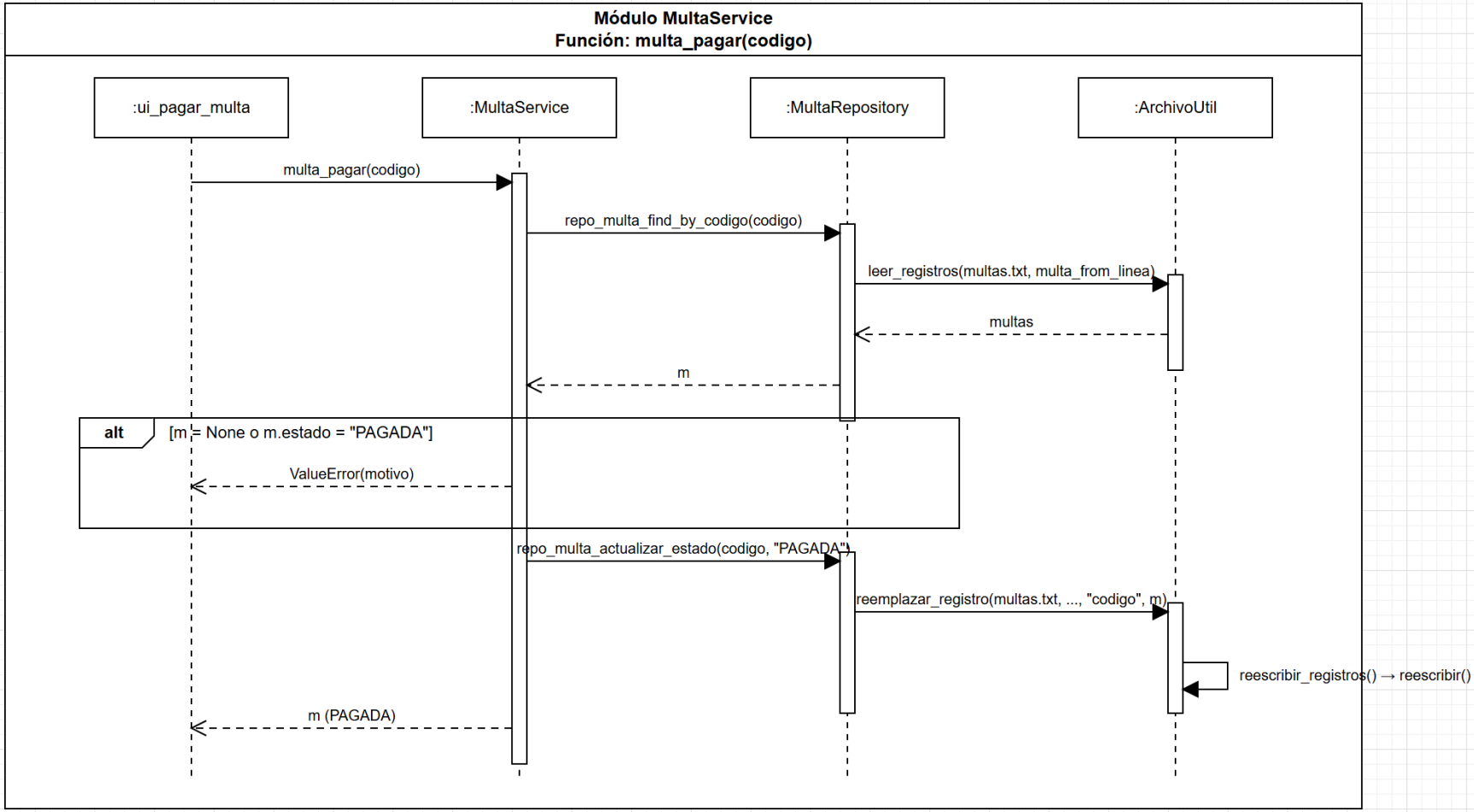


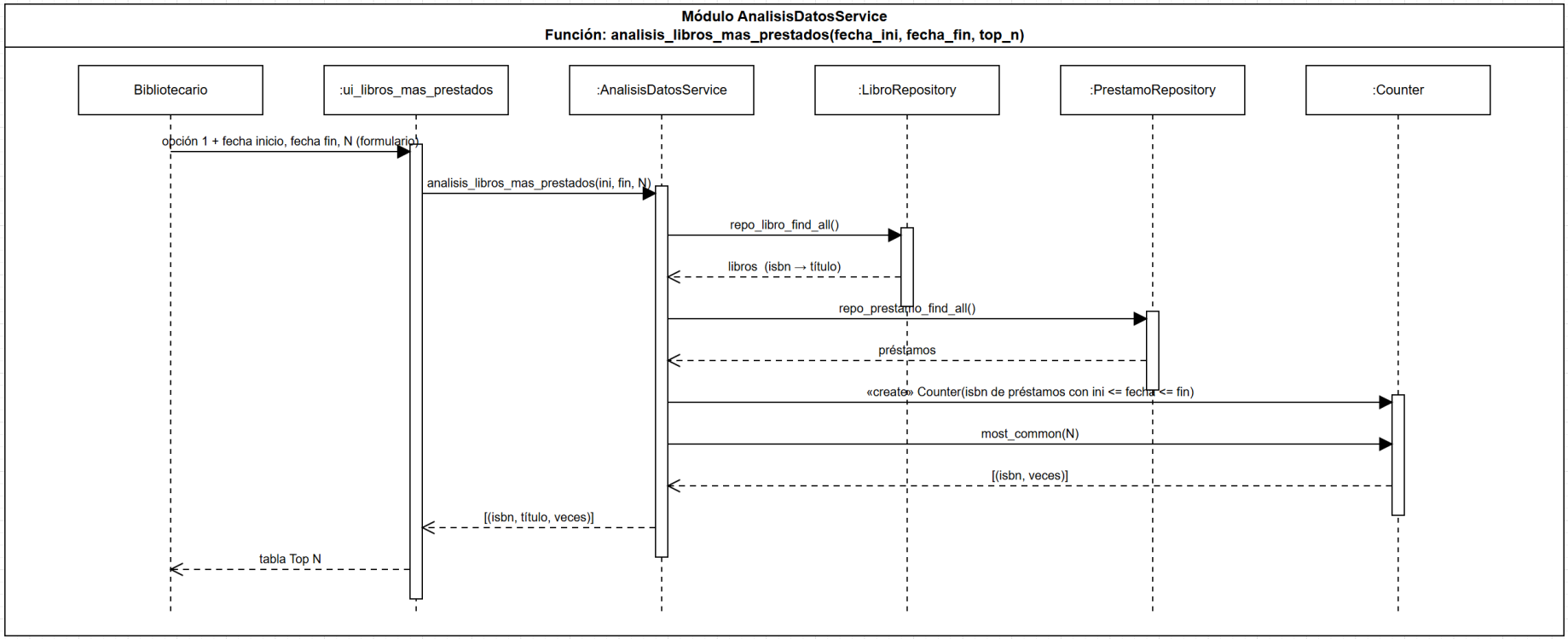

Módulo Interfaz (menu_usuarios)
Función: ui_registrar_usuario() → leer_formulario()
Bibliotecario
:ui_registrar_usuario
:leer_formulario
:leer_campo
:validadores v_*
:UsuarioService
:UsuarioRepository
opción 1: Registrar usuario
leer_formulario("Registrar usuario", 5 campos)
loop
[mientras i < 5 campos]
leer_campo(campos[i], valores, i+1, 5, prev|)
loop
[hasta que el dato sea válido]
"[i/5] Número de documento (5-12 dígitos) "
texto
alt
[texto = "<" o "*"]
"<" (i = i - 1) / "*" (cancelar → None)
[dato]
validar(texto, valores) p. ej. v_documento_nuevo()
valor o ValueError(motivo)
"✗ motivo. Corrija el dato" (solo si es inválido)
valor válido
valores[clave] = valor ; i = i + 1
Resumen 1..5 "¿Guardar? (s / número / < / *)"
"s" (o el número del campo a corregir)
valores (None si se canceló)
opt
[valores ≠ None]
usuario_registrar(doc, nombre, tipo, correo, tel)
crear_usuario(...) (valida de nuevo)
repo_usuario_find_by_documento(doc)
None
repo_usuario_guardar(u) → usuarios.txt
"Usuario <nombre> registrado."

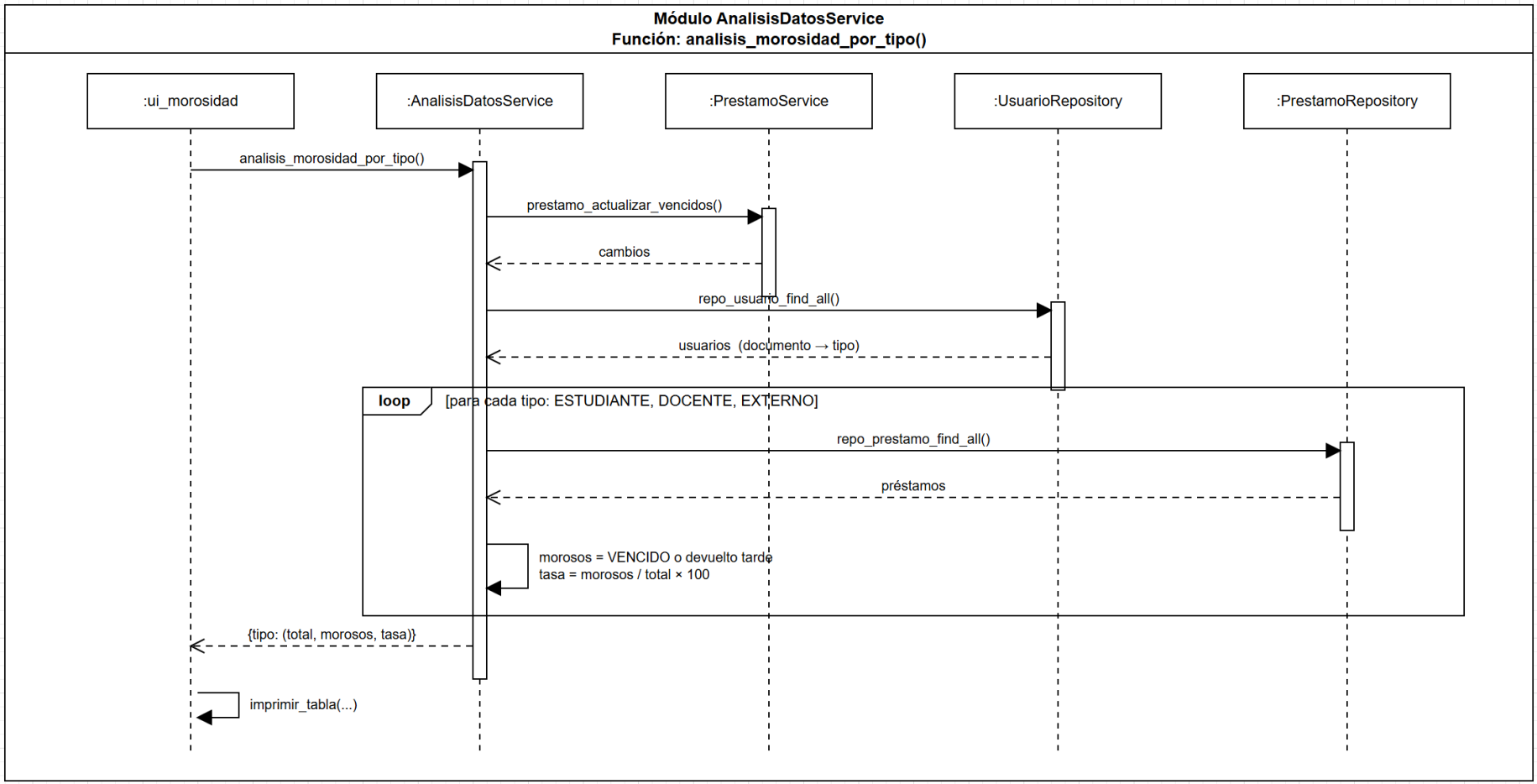

Módulo PrestamoService
Función: prestamo_prestar(num_documento, isbn, fecha)
:ui_prestar
:PrestamoService
:UsuarioService
:MultaService
:LibroService
:ReservaService
:PrestamoRepository
:LibroRepository
:ReservaRepository
prestamo_prestar(doc, isbn, fecha)
usuario_buscar_por_documento(doc)
usuario
multa_listar_pendientes_usuario(doc)
multas pendientes
libro_buscar_por_isbn(isbn)
libro
prestamo_actualizar_vencidos()
repo_prestamo_find_by_usuario(doc)
préstamos abiertos
alt
[suspendido, con multas, cupo agotado, ya lo tiene o sin ejemplares]
ValueError(motivo)
reserva_listar_por_libro(isbn)
cola de reservas ACTIVAS (orden de llegada)
alt
[el usuario tiene turno en la cola]
repo_reserva_actualizar_estado(cod, "CUMPLIDA")
[ejemplares apartados para otros usuarios]
ValueError("apartados por reservas")
codigo = generar_codigo("PRE")
fecha_limite = fecha + días del tipo
repo_prestamo_guardar(p) → prestamos.txt
repo_libro_actualizar(libro) (disponibles - 1)
p: Prestamo


Módulo PrestamoService
Función: prestamo_devolver(codigo, fecha)
:ui_devolver
:PrestamoService
:PrestamoRepository
:LibroRepository
:MultaService
:MultaRepository
prestamo_devolver(codigo, fecha)
repo_prestamo_find_by_codigo(codigo)
p
alt
[no existe, ya DEVUELTO o fecha inválida]
ValueError(motivo)
p.fecha_devolucion = fecha
p.estado = "DEVUELTO"
repo_prestamo_actualizar(p)
repo_libro_find_by_isbn(isbn)
libro
repo_libro_actualizar(libro) (disponibles + 1)
multa_calcular(p)
dias = fecha_devolucion - fecha_limite
alt
[dias > 0]
valor = dias × 1.000
repo_multa_guardar(multa PENDIENTE) → multas.txt
multa
[entregado a tiempo]
None
(p, multa)


Módulo ReservaService
Función: reserva_reservar(num_documento, isbn, fecha)
:ui_reservar
:ReservaService
:UsuarioService
:LibroService
:PrestamoRepository
:ReservaRepository
reserva_reservar(doc, isbn)
usuario_buscar_por_documento(doc)
usuario
libro_buscar_por_isbn(isbn)
libro
cola = reserva_listar_por_libro(isbn)
repo_prestamo_find_by_usuario(doc)
préstamos del usuario
alt
[suspendido, disponibles > cola, ya en la cola o ya lo tiene]
ValueError(motivo)
r = crear_reserva(generar_codigo("RES"),
isbn, doc, hoy, "ACTIVA")
repo_reserva_guardar(r) → reservas.txt
r: Reserva
"Reserva RES-... creada. Puesto en la cola: n"


Módulo MultaService
Función: multa_pagar(codigo)
:ui_pagar_multa
:MultaService
:MultaRepository
:ArchivoUtil
multa_pagar(codigo)
repo_multa_find_by_codigo(codigo)
leer_registros(multas.txt, multa_from_linea)
multas
m
alt
[m = None o m.estado = "PAGADA"]
ValueError(motivo)
repo_multa_actualizar_estado(codigo, "PAGADA")
reemplazar_registro(multas.txt, ..., "codigo", m)
reescribir_registros() → reescribir()
m (PAGADA)


Módulo AnalisisDatosService
Función: analisis_libros_mas_prestados(fecha_ini, fecha_fin, top_n)
Bibliotecario
:ui_libros_mas_prestados
:AnalisisDatosService
:LibroRepository
:PrestamoRepository
:Counter
opción 1 + fecha inicio, fecha fin, N (formulario)
analisis_libros_mas_prestados(ini, fin, N)
repo_libro_find_all()
libros (isbn → título)
repo_prestamo_find_all()
préstamos
«create» Counter(isbn de préstamos con ini <= fecha <= fin)
most_common(N)
[(isbn, veces)]
[(isbn, título, veces)]
tabla Top N


Módulo AnalisisDatosService
Función: analisis_morosidad_por_tipo()
:ui_morosidad
:AnalisisDatosService
:PrestamoService
:UsuarioRepository
:PrestamoRepository
analisis_morosidad_por_tipo()
prestamo_actualizar_vencidos()
cambios
repo_usuario_find_all()
usuarios (documento → tipo)
loop
[para cada tipo: ESTUDIANTE, DOCENTE, EXTERNO]
repo_prestamo_find_all()
préstamos
morosos = VENCIDO o devuelto tarde
tasa = morosos / total × 100
{tipo: (total, morosos, tasa)}
imprimir_tabla(...)